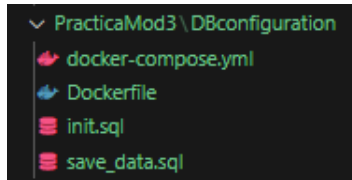


Semana 8. Práctica 3C. Gestión de una base de datos.

1. Introducción.

Se realiza la configuración del contenedor de Docker y se crea la DB (base de datos), utilizando los archivos que se fueron creando durante la práctica. Los archivos creados fueron:

- docker-compose.yml
- Dockerfile
- save_data.sql
- init.sql



2. Desarrollo.

Los pasos que realizamos fueron los siguientes:

1. Creación del **docker-compose.yml**.

```
docker-compose.yml U Dockerfile U save_data.sql U init.sql U
BsDa > Postgresql > Docker > PracticaMod3 > DBconfiguration > docker-compose.yml
  > Run All Services
1 services:
  > Run Service
2 postgres:
3   build: . #Construir contenedor usando Dockerfile presente en el directorio actual (.)
4   container_name: ContDBpractMod3 # Nombre del contenedor
5   ports:
6     - "5432:5432" # Puerto para conectar desde tu máquina host al contenedor
7   environment:
8     POSTGRES_USER: Admin # Usuario de la base de datos
9     POSTGRES_PASSWORD: p4ssw0rdDB # Contraseña del usuario
10    POSTGRES_DB: credenciales # Nombre de la base de datos
11   volumes:
12     - postgres_data:/var/lib/postgresql/data # Espacio donde los datos se conservarán, al detener el contenedor.
13
14 volumes:
15   postgres_data:
16     driver: local
17
```

2. Creación del **Dockerfile**.

```
docker-compose.yml U Dockerfile U X save_data.sql U init.sql U
BsDa > Postgresql > Docker > PracticaMod3 > DBconfiguration > Dockerfile > ...
1 # Usa la imagen oficial de PostgreSQL
2 FROM postgres:latest
3
4 # Copia el script SQL al directorio de inicialización de PostgreSQL
5 COPY init.sql /docker-entrypoint-initdb.d/
6 COPY save_data.sql /docker-entrypoint-initdb.d/
7
8 # PostgreSQL ejecutará automáticamente los scripts en /docker-entrypointinitdb.d/
```

3. Creación del **init.sql**: script para crear las tablas usuarios y credenciales.

```
docker-compose.yml U Dockerfile U X save_data.sql U init.sql U X
BsDa > Postgresql > Docker > PracticaMod3 > DBconfiguration > init.sql
  Connect to MSSQL
1  -- Crear la tabla para almacenar datos de usuarios
2  CREATE TABLE usuarios
3  (
4      id_usuario SERIAL PRIMARY KEY,
5      nombre VARCHAR(100) NOT NULL,
6      correo VARCHAR(255) UNIQUE,
7      telefono VARCHAR(15),
8      fecha_nacimiento DATE
9  );
10 -- Crear la tabla para almacenar usuarios y contraseñas
11 CREATE TABLE credenciales
12 (
13     id_credencial SERIAL PRIMARY KEY,
14     id_usuario INT NOT NULL,
15     username VARCHAR(50) UNIQUE NOT NULL,
16     password_hash VARCHAR(255) NOT NULL,
17     FOREIGN KEY (id_usuario) REFERENCES usuarios (id_usuario)
18 );
```

4. Creación del **save_data.sql**: instrucciones INSERT para poblar las tablas con 20 registros.

```
docker-compose.yml U Dockerfile U save_data.sql U X init.sql U
BsDa > Postgresql > Docker > PracticaMod3 > DBconfiguration > save_data.sql
  Connect to MSSQL
1  -- Insertar datos en la tabla usuarios
2  INSERT INTO usuarios
3      (nombre, correo, telefono, fecha_nacimiento)
4  VALUES
5      ('Juan Pérez', 'juan.perez1@example.com', '1234567890', '1985-01-15'),
6      ('Ana Gómez', 'ana.gomez2@example.com', '1234567891', '1990-03-22'),
7      ('Luis Martínez', 'luis.martinez3@example.com', '1234567892', '1988-07-10'),
8      ('María López', 'maria.lopez4@example.com', '1234567893', '1992-11-05'),
9      ('Carlos Ruiz', 'carlos.ruiz5@example.com', '1234567894', '1980-06-25'),
10     ('Sofía Castro', 'sofia.castro6@example.com', '1234567895', '1995-02-18'),
11     ('David Ramírez', 'david.ramirez7@example.com', '1234567896', '1983-09-09'),
12     ('Elena Ortega', 'elena.ortega8@example.com', '1234567897', '1991-05-03'),
13     ('Miguel Torres', 'miguel.torres9@example.com', '1234567898', '1993-12-14'),
14     ('Laura Sánchez', 'laura.sanchez10@example.com', '1234567899', '1987-08-01'),
15     ('Pedro Morales', 'pedro.morales11@example.com', '1234567800', '1986-04-17'),
16     ('Clara Hernández', 'clara.hernandez12@example.com', '1234567801', '1994-06-30'),
17     ('Jorge Rojas', 'jorge.rojas13@example.com', '1234567802', '1981-03-25'),
18     ('Valeria Peña', 'valeria.pena14@example.com', '1234567803', '1992-01-09'),
19     ('Andrés Romero', 'andres.romero15@example.com', '1234567804', '1989-10-12'),
20     ('Camila Paredes', 'camila.paredes16@example.com', '1234567805', '1997-11-19'),
21     ('Ricardo Vargas', 'ricardo.vargas17@example.com', '1234567806', '1984-07-23'),
22     ('Daniela Flores', 'daniela.flores18@example.com', '1234567807', '1996-03-01'),
23     ('Héctor Serrano', 'hector.serrano19@example.com', '1234567808', '1982-02-11'),
24     ('Patricia Vega', 'patricia.vega20@example.com', '1234567809', '1990-09-05');
25
```

```
docker-compose.yml U Dockerfile U save_data.sql X init.sql U
BsDa > Postgresql > Docker > PracticaMod3 > DBconfiguration > save_data.sql
4 VALUES
23 ('hector.serrano', 'hector.serrano19@example.com', '1234567808', '1982-02-11'),
24 ('Patricia Vega', 'patricia.vega20@example.com', '1234567809', '1990-09-05');
25
26 -- Insertar datos en la tabla credenciales
27 INSERT INTO credenciales
28 (id_usuario, username, password_hash)
29 VALUES
30 (1, 'juan.perez1', 'hash_juan_perez'),
31 (2, 'ana.gomez2', 'hash_ana_gomez'),
32 (3, 'luis.martinez3', 'hash_luis_martinez'),
33 (4, 'maria.lopez4', 'hash_maria_lopez'),
34 (5, 'carlos.ruiz5', 'hash_carlos_ruiz'),
35 (6, 'sofia.castro6', 'hash_sofia_castro'),
36 (7, 'david.ramirez7', 'hash_david_ramirez'),
37 (8, 'elena.ortega8', 'hash_elena_ortega'),
38 (9, 'miguel.torres9', 'hash_miguel_torres'),
39 (10, 'laura.sanchez10', 'hash_laura_sanchez'),
40 (11, 'pedro.morales11', 'hash_pedro_morales'),
41 (12, 'clara.hernandez12', 'hash_clara_hernandez'),
42 (13, 'jorge.rojas13', 'hash_jorge_rojas'),
43 (14, 'valeria.pena14', 'hash_valeria_pena'),
44 (15, 'andres.romero15', 'hash_andres_romero'),
45 (16, 'camila.paredes16', 'hash_camila_paredes'),
46 (17, 'ricardo.vargas17', 'hash_ricardo_vargas'),
47 (18, 'daniela.flores18', 'hash_daniela_flores'),
48 (19, 'hector.serrano19', 'hash_hector_serrano'),
49 (20, 'patricia.vega20', 'hash_patricia_vega');
50
```

5. Se inicio Docker y usa la terminal o Powershell para crear el contenedor y la base de dato. Se utiliza el comando:

docker-compose up -d

6. Se valido que la base de datos se haya creado correctamente. Se utilizan los siguientes comandos:

docker exec -it ContDBpractMod3 bash

psql -U Admin -d credenciales

SELECT * FROM usuarios;

En este paso, no se mostraban registros.

```
Administrador: Windows PowerShell
PS C:\Users\phurtado\Documents\2025\Curso BTSP\BsDa\Postgresql\Docker\PracticaMod3\DBconfiguration> docker exec -it ContDBpractMod3 bash
root@242110b60b19:/# psql -U Admin -d credenciales
psql (17.5 (Debian 17.5-1.pgdg120+1))
Type "help" for help.

credenciales=# SELECT * FROM usuarios;
 id_usuario | nombre | correo | telefono | fecha_nacimiento
-----
(0 rows)
```

Al parecer no se ejecuto de manera automática el script save_data.sql. tuve que realizarlo de manera manual con el siguiente comando:

docker cp save_data.sql ContDBpractMod3:/save_data.sql

7. Con la ejecución manual, se resolvió el detalle y después se realiza la consulta:

SELECT * FROM usuarios;

Mostrando el resultado de los 20 registros insertados.

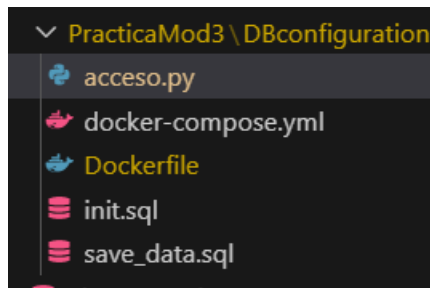
```
Administrador: Windows PowerShell

INSERT 0 20
root@62c53a4f3581:/# psql -U Admin -d credenciales
psql (17.5 (Debian 17.5-1.pgdg120+1))
Type "help" for help.

credenciales=# SELECT * FROM usuarios;
 id_usuario | nombre | correo | telefono | fecha_nacimiento
-----+-----+-----+-----+-----
1 | Juan Pérez | juan.perez1@example.com | 1234567890 | 1985-01-15
2 | Ana Gómez | ana.gomez2@example.com | 1234567891 | 1990-03-22
3 | Luis Martínez | luis.martinez3@example.com | 1234567892 | 1988-07-10
4 | María López | maria.lopez4@example.com | 1234567893 | 1992-11-05
5 | Carlos Ruiz | carlos.ruiz5@example.com | 1234567894 | 1980-06-25
6 | Sofía Castro | sofia.castro6@example.com | 1234567895 | 1995-02-18
7 | David Ramírez | david.ramirez7@example.com | 1234567896 | 1983-09-09
8 | Elena Ortega | elena.ortega8@example.com | 1234567897 | 1991-05-03
9 | Miguel Torres | miguel.torres9@example.com | 1234567898 | 1993-12-14
10 | Laura Sánchez | laura.sanchez10@example.com | 1234567899 | 1987-08-01
11 | Pedro Morales | pedro.morales11@example.com | 1234567800 | 1986-04-17
12 | Clara Hernández | clara.hernandez12@example.com | 1234567801 | 1994-06-30
13 | Jorge Rojas | jorge.rojas13@example.com | 1234567802 | 1981-03-25
14 | Valeria Peña | valeria.pena14@example.com | 1234567803 | 1992-01-09
15 | Andrés Romero | andres.romero15@example.com | 1234567804 | 1989-10-12
16 | Camila Paredes | camila.paredes16@example.com | 1234567805 | 1997-11-19
17 | Ricardo Vargas | ricardo.vargas17@example.com | 1234567806 | 1984-07-23
18 | Daniela Flores | daniela.flores18@example.com | 1234567807 | 1996-03-01
19 | Héctor Serrano | hector.serrano19@example.com | 1234567808 | 1982-02-11
20 | Patricia Vega | patricia.vega20@example.com | 1234567809 | 1990-09-05
(20 rows)
```

3. Etapa 2. Transacciones a la Base de Datos.

- Se crea el archivo acceso.py desde el VScode.



```
docker-compose.yml Dockerfile 1 save_data.sql init.sql acceso.py M X
BsDa > Postgresql > Docker > PracticaMod3 > DBconfiguration > acceso.py > ...
1 import psycopg2
2 import getpass
3
4 # Configuración de conexión a la base de datos en Docker
5 DB_HOST = "localhost"
6 DB_PORT = "5432"
7 DB_NAME = "credenciales"
8 DB_USER = 'Admin'
9 DB_PASSWORD = "p4ssw0rdDB"
10
11 def conectar_db():
12     """Conecta a la base de datos PostgreSQL y retorna la conexión."""
13     try:
14         conn = psycopg2.connect(
15             host=DB_HOST,
16             port=DB_PORT,
17             database=DB_NAME,
18             user=DB_USER,
19             password=DB_PASSWORD
20         )
21         return conn
22     except Exception as e:
```

- Ejecución del archivo acceso.py con el comando `python .\acceso.py`

```
PS C:\Users\pedro.hurtado\OneDrive -  
.py  
Inicio de sesión en la base de datos  
Ingrese su usuario: sofia.castro6  
Ingrese su contraseña:  
  
Datos del usuario encontrado:  
ID: 6  
Nombre: Sofía Castro  
Correo: sofia.castro6@example.com  
Teléfono: 1234567895  
Fecha de Nacimiento: 1995-02-18
```

```
Inicio de sesión en la base de datos  
Ingrese su usuario: andres.romero15  
Ingrese su contraseña:  
  
Datos del usuario encontrado:  
ID: 15  
Nombre: Andrés Romero  
Correo: andres.romero15@example.com  
Teléfono: 1234567804  
Fecha de Nacimiento: 1989-10-12
```

- Función para ingresar (insertar) nuevos usuarios a la base de datos - credenciales.

```
def insertar_nvo_usuario(nombre, correo, telefono, fecha_nacimiento):  
    conn = conectar_db()  
    if not conn:  
        return  
    try:  
        cursor = conn.cursor()  
        query = """  
            INSERT INTO usuarios  
            (nombre, correo, telefono, fecha_nacimiento)  
            VALUES  
            (%s, %s, %s,%s)  
        """  
        cursor.execute(query, (nombre, correo, telefono, fecha_nacimiento))  
        conn.commit()  
        cursor.close()  
        conn.close()  
        print("Se insertaron los datos correctamente")
```

```
print("Inserte los datos del nuevo usuario")  
nombre = input("Ingrese el nombre del nuevo usuario: ")  
correo = input("Ingrese el correo del nuevo usuario: ")  
telefono = input("Ingrese el telefono del nuevo usuario: ")  
fecha_nacimiento = input("Ingrese la fecha de nacimiento del nuevo usuario: ")  
  
insertar_nvo_usuario(nombre, correo, telefono, fecha_nacimiento)
```

```

Ingrese el nombre del nuevo usuario: Pedro Hurtado
Ingrese el correo del nuevo usuario: pedro.hurtado@prueba.com
Ingrese el telefono del nuevo usuario: 2810198440
Ingrese la fecha de nacimiento del nuevo usuario: 1984-10-28
Se insertaron los datos correctamente

```

```

credenciales=# select * from usuarios;

```

id_usuario	nombre	correo	telefono	fecha_nacimiento
1	Juan Pérez	juan.perez1@example.com	1234567890	1985-01-15
2	Ana Gómez	ana.gomez2@example.com	1234567891	1990-03-22
3	Luis Martínez	luis.martinez3@example.com	1234567892	1988-07-10
4	María López	maria.lopez4@example.com	1234567893	1992-11-05
5	Carlos Ruiz	carlos.ruiz5@example.com	1234567894	1980-06-25
6	Sofía Castro	sofia.castro6@example.com	1234567895	1995-02-18
7	David Ramírez	david.ramirez7@example.com	1234567896	1983-09-09
8	Elena Ortega	elena.ortega8@example.com	1234567897	1991-05-03
9	Miguel Torres	miguel.torres9@example.com	1234567898	1993-12-14
10	Laura Sánchez	laura.sanchez10@example.com	1234567899	1987-08-01
11	Pedro Morales	pedro.morales11@example.com	1234567800	1986-04-17
12	Clara Hernández	clara.hernandez12@example.com	1234567801	1994-06-30
13	Jorge Rojas	jorge.rojas13@example.com	1234567802	1981-03-25
14	Valeria Peña	valeria.pena14@example.com	1234567803	1992-01-09
15	Andrés Romero	andres.romero15@example.com	1234567804	1989-10-12
16	Camila Paredes	camila.paredes16@example.com	1234567805	1997-11-19
17	Ricardo Vargas	ricardo.vargas17@example.com	1234567806	1984-07-23
18	Daniela Flores	daniela.flores18@example.com	1234567807	1996-03-01
19	Héctor Serrano	hector.serrano19@example.com	1234567808	1982-02-11
20	Patricia Vega	patricia.vega20@example.com	1234567809	1990-09-05
23	Diana Colin	diana.colin@prueba.com	240720025	2000-07-24
26	Diana Colin 2	diana.colin2@prueba.com	12347890	2000-07-24
27	Pedro Hurtado	pedro.hurtado@prueba.com	2810198440	1984-10-28

(23 rows)

- Para el siguiente planteamiento:

“Notarás que la base de datos tiene el mismo nombre que una de sus tablas, credenciales. Este problema puede afectar la consistencia y la navegabilidad. ¿Cómo puedes solucionar este problema de diseño?”

Posible Solución: Para mejorar la consistencia y navegabilidad del diseño de la base de datos, se propondría cambiar el nombre de la tabla “credenciales” a “autenticaciones” o “accesos”. Esto se debe a que el nombre actual genera confusión con el nombre de la base de datos.

El nuevo nombre 'autenticaciones' es más específico y descriptivo, ya que refleja con precisión el propósito de la tabla, que es almacenar información relacionada con el proceso de autenticación de usuarios.

- Para el último planteamiento:

“Imagina que esta base de datos se utilizará para el control de trabajadores de una institución. Agrega una tabla con la información relacionada a sus puestos de trabajo y asóciala al registro del personal. Si es necesario realiza ajustes en el diseño de la base de datos.”

Posible Solución: Se propondría crear una nueva tabla llamada "puestos_trabajo" y agregar una columna a la tabla "usuarios" para guardar el ID del puesto de trabajo asignado a cada usuario.

```
CREATE TABLE puestos_trabajo (  
    id_puesto SERIAL PRIMARY KEY,  
    nombre_puesto VARCHAR(100) NOT NULL,  
    descripcion TEXT  
);
```

```
ALTER TABLE usuarios  
ADD COLUMN id_puesto INT;
```
